

Plan of Attack:

For our plan of attack, we will follow the advice section given within the guidelines step by step.

We're going to use the MVC architecture, where we will have a Game class in charge of orchestrating the gameplay and the models described in the UML diagrams and a textual View. For Game mutations, we opted for the Command pattern, where each card which mutates the Game issues a command to the Game, which is then executed by the Game.

Minions

We opted to make Minion an abstract class, and each minion a concrete class of its own. We agreed on this pattern to make the development process less error-prone, instead of creating AirElementals by having `new Minion("AirElemental", 1, 1)`. We have `new AirElemental()` where the concrete class overrides the attack and defense stats. This keeps our code free of repetition and allows us to make new minions easily. Each concrete class will have private fields for the description, name, etc. Minions with triggered abilities implement the method `trigger()` which will activate the triggered ability if the right trigger is passed. Attacks and abilities will be facilitated through Commands issued to the Game.

Enchantments

For Enchantments (as discussed below), we will use the decorator pattern and each concrete class for enchantments (not fully listed in the UML, we only show Enrage), more information is answered below. Enchantments will implement the `getAttack()` and such methods, modifying the base minions values based on the specific Enchantment.

Rituals

Rituals will be implemented as cards, and is an abstract class and each concrete ritual will implement the `use` method, which will decrease the charges by the cost of the triggered ability. All cards have a `trigger(Trigger)` method which will be implemented on the ritual to call `use` if the Trigger is for the triggered ritual.

Card Collection

We have abstracted Graveyard, Board and Hand to CardCollections which hold a number of cards. The Board will have the trigger method which will be called on all cards, Hand will implement a method, isFull and each card collection will facilitate addition, removal and retrieval of Card pointers. The hand will take a file path where to get the deck (default being "default.deck")

Spells

Spells are implemented by having each spell implement a command and when a spell is cast, we notify the Game with the Command to be executed and it then calls `Command.execute(*this)`

Abilities

At the four trigger times, the game will call a `trigger(Trigger(enum))` method, which passes through an enum value of the current Trigger. This will then be passed through the player and the board, checking each card on the player's board for if a trigger exists and if a trigger needs to be called. If yes, then the `use()` on the triggered ability gets called. Each triggered ability is its own concrete class, inheriting from the TriggeredAbility abstract class and the Ability abstract class.

Activated abilities will also have their own concrete class, inheriting from an ActivatedAbility abstract class and the Ability abstract class. When an activated ability is used, the `use()` method on a minion card will call the `use()` method on the activated ability object, which will control the functionality.

View and Textual View

We have an abstract view class which has a generic method, `display`, which is meant to display the Game using all the public methods. The textual view prints to standard output but with this design there could even be a graphical ui.

Game Controller

The game controller orchestrates most of the interactions of the game. If “-init” is passed, the game controller calls init to process the commands from the file stream.

Initialization (Basic Gameplay)

The basic gameplay will all be handled within playGame() in the Game controller.

help command

Game controller will call the help method in View which will display the information.

end

The game controller will switch active players, effectively switching the turn.

quit

The playGame() while loop will end, ending the gameplay.

hand

The game controller will call getHand() on the active player and calls
view→showHand(const Hand&)

board

The game controller will call getBoard() on the active player and then call
view→showBoard(const Board&).

play

The game controller will call getCard(i) on the hand of the active player, then playCard(*Card) on that card. Each card has a use() method that controls the card's functionality and any abilities that may be associated with it. If an ability or spell needs global access, it can notify the game controller, which will then handle that functionality through the issued command.

If i, p, and t are all provided, getCard(t) can be called on the player p, returning the card and all of the getters/setters that may be needed given the functionality of the card i.

discard

The Game controller will get the active players hand, call removeCard(i) and add the removed card to the players graveyard.

attack

If no j is provided, the Game gets the hand of the active player, gets the minion at index i And then orchestrates the attack on the non active player by decreasing the health points of the player by the minions attack.

If another minion index is provided, the game gets the non active players hand and orchestrates the attack between the two minions, setting eaches defense appropriately. If a minion's defence goes below 0, the game removes the minion from the hand of the player who owns it and adds it to that players graveyard.

inspect

The view will go through each enchantment on the minion, adding it to a stack of enchantments then once the it reaches the base minion, it will output this to whatever view it is and then print all the enchantments from the stack. The Game calls view->inspect(const Minion&)

General Plan, Questions & Notes (for self reflection)

We plan on implementing the skeleton functionality by Friday, July 18. We then plan on implementing the rest of the functionality by Monday, July 21. By setting our goal ahead of time, we can ensure that we will have time to address any random issues that might occur. We then will spend the rest of the time before the deadline writing the report and potentially implementing any extra credit features. We plan on doing the challenge enhancement of manually managing all memory using RAI and with minimal raw pointers. We expect plan changes as we implement the Game as is the nature of any large project

We all equally contributed to the UML diagram, working in person. We plan on having a shared git where we can all contribute to the repository with the work that we have done. We will split the steps 2 to 10 equally, with each person working on the next step entirely when they can (take 3 steps each). The other two people in the group will be responsible for reviewing the code for each step before finalizing. Steps 11-12 will then be done in person together along with the report.

Questions:

How could you design activated abilities in your code to maximize code reuse?

We can design activated ability as an abstract class, with each unique activated ability having its own concrete class with its unique description/functionality. This ensures that the code common to any activated ability (ex. `getCost()`) is reused for each one and does not need to be repeated.

For instance, "Deal 1 damage to target minion" will be implemented as a concrete subclass of activated ability. In the future, if there are additional activated abilities, they can be implemented easily with this pattern.

What design pattern would be ideal for implementing enchantments? Why?

The ideal design pattern is decorators. The reasoning is that it makes stacking enchantments (modifications) on minions simple and handles any additional features to minions that need to be added for code flexibility. In our case, we will be implementing the enchantment functionality as minion decorators, so each enchantment will represent a unique decorator that can be applied to a base minion object.

Suppose we found a solution to the space limitations of the current user interface and wanted to allow minions to have any number and combination of activated and triggered abilities. What design patterns might help us achieve this while maximizing code reuse?

We can have a triggered ability abstract class and an activated ability abstract class which inherit from an ability abstract class, with each unique ability being a concrete subclass. This ensures that the common functionalities between any two abilities will be shared. Additionally, the way abilities are called within the Minion class will be written in a way so that any ability can be called with the same method without affecting correctness. This ensures that code will not be duplicated. And we allow the minion to have a vector of unique pointers to Abilities and add a boolean of `is_triggered()` to differentiate between triggered abilities and activated abilities. The command pattern will help with ability actions.